Documentazione Tecnica del Progetto: Intorni K-pop

Questa documentazione descrive in dettaglio l'architettura, le strutture dati, gli algoritmi e le scelte di progettazione alla base dell'applicazione **Intorni K-pop**.

1. Panoramica del Progetto e Come è Stato Creato

1.1 Obiettivo dell'Applicazione

L'applicazione permette di esplorare interattivamente la complessa rete di relazioni tra artisti, gruppi musicali e agenzie discografiche (label) del panorama K-pop.
Invece di visualizzare l'intero grafo contemporaneamente — operazione che risulterebbe illeggibile e pesante computazionalmente — l'applicazione adotta il paradigma dell'**analisi di ego-network / intorno mirato**:

- L'utente seleziona uno o più nodi di partenza (**seed**).
- Definisce i parametri di espansione (raggio massimo di esplorazione, trattamento delle agenzie).
- Identifica ed esclude nodi di disturbo o hub dominanti (**sfavoriti / blocked**).
- Esplora i collegamenti, evidenziando i nodi che fungono da ponte (**bridge**) tra più partenze.

1.2 Origine dei Dati e Nascita del Progetto

1. **Dati originali**:
    - [K-pop_node.csv](file:///D:/Users/yyi30043/Desktop/VDI/K-pop_node.csv): elenco dei nodi contenente ID, tipo e denominazione.
    - [K-pop_edge.csv](file:///D:/Users/yyi30043/Desktop/VDI/K-pop_edge.csv): lista di adiacenza degli archi non orientati tra entità.
2. **Dal prototipo monolitico all'architettura modulare**:
    - Inizialmente sviluppata come prototipo monolitico all-in-one in [kpop_intorni.html](file:///D:/Users/yyi30043/Desktop/VDI/kpop_intorni.html), l'applicazione è stata successivamente rifattorizzata in un'architettura modulare moderna in **Vanilla JavaScript (ES Modules)** senza necessità di bundler (Webpack, Vite, ecc.) né fasi di compilazione.

- Il dataset CSV è stato pre-elaborato nel formato compatto [data.json](file:///D:/Users/yyi30043/Desktop/VDI/src/data/data.json), riducendo l'overhead di parsing a runtime.

1.3 Struttura dei Moduli

L'applicazione segue il principio di separazione netta tra logica algoritmica pura e presentazione:

```
VDI/
├── index.html                # Scheletro HTML dell'interfaccia
├── styles.css                # Foglio di stile unico con supporto automatico Dark Mode
├── vendor/
│   └── d3.v7.min.js          # D3.js (utilizzato per simulazione a forze, zoom e drag)
├── src/
│   ├── data/
│   │   └── data.json         # Array compatti [id, tipo, nome] e archi [u, v]
│   ├── core/                 # Logica pura (indipendente dal DOM, testabile in Node.js)
│   │   ├── graph.js          # Normalizzazione dati, costruzione indici e liste di adiacenza
│   │   ├── neighborhood.js   # BFS multi-seed, calcolo intorni e percorsi minimi
│   │   └── search.js         # Motore di ricerca testuale con normalizzazione diacritici
│   └── ui/                   # Livello di presentazione e interazione utente
│       ├── state.js          # Store reattivo (pattern Pub/Sub unificatore dello stato)
│       ├── palette.js        # Lettura colori dalle variabili CSS e ascolto Dark Mode
│       ├── dom-helpers.js    # Utility per la creazione dichiarativa di elementi DOM
│       ├── canvas-renderer.js  # Motore grafico su HTML5 Canvas e simulazione D3-Force
│       ├── sidebar.js        # Gestione input di ricerca, filtri, statistiche e hub
│       └── detail-panel.js   # Scheda informativa del nodo selezionato
└── test/                     # Test unitari per il core eseguiti con node:test
```

2. Calcolo dei Vicini e dell'Intorno

Il calcolo dell'intorno è affidato alle funzioni pure implementate in [src/core/neighborhood.js](file:///D:/Users/yyi30043/Desktop/VDI/src/core/neighborhood.js).

2.1 L'Algoritmo `computeNeighborhood`

La funzione `computeNeighborhood({ adj, typeOf, seeds, blocked, radius, labelMode })` implementa una **ricerca in ampiezza (BFS - Breadth-First Search) multi-sorgente**.

```
export function computeNeighborhood({ adj, typeOf, seeds, blocked, radius, labelMode }) {
  const isLabel = id => typeOf(id) === 'label';
  const hop = new Map();
  const reachedBy = new Map();

  for (const s of seeds) {
    const dist = new Map([[s, 0]]);
    let frontier = [s];
    for (let d = 1; d <= radius && frontier.length; d++) {
      const next = [];
      for (const u of frontier) {
        if (labelMode === 'endpoint' && u !== s && isLabel(u)) continue;
        for (const v of adj.get(u)) {
          if (dist.has(v) || blocked.has(v)) continue;
          if (labelMode === 'hide' && isLabel(v)) continue;
          dist.set(v, d);
          next.push(v);
        }
      }
      frontier = next;
    }
    for (const [v, d] of dist) {
      if (!hop.has(v) || d < hop.get(v)) hop.set(v, d);
      if (!reachedBy.has(v)) reachedBy.set(v, []);
      reachedBy.get(v).push(s);
    }
  }

  const edges = [];
  for (const u of hop.keys())
    for (const v of adj.get(u))
      if (u < v && hop.has(v)) edges.push([u, v]);

  return { hop, reachedBy, edges };
}
```

Come funziona passo per passo:

1. **Iterazione per ciascun Seed**:
   - Per ogni nodo $s \in \text{seeds}$, viene avviata una BFS a livelli.
   - `dist` memorizza la distanza minima dal seed corrente: $\text{dist}(s) = 0$.

- `frontier` mantiene i nodi del livello corrente $d-1$, espandendo verso il livello successivo d fino al raggiungimento di `radius` .

2. **Gestione dei Nodi Bloccati (`blocked`)**:
   - Se un nodo v appartiene al set `blocked` , viene scartato immediatamente (`blocked.has(v)`). Questo interrompe qualsiasi ulteriore propagazione attraverso quel ramo.
3. **Gestione delle Agenzie (`labelMode`)**:
   - Se `labelMode === 'hide'` : i nodi di tipo `label` non vengono mai aggiunti (`if (labelMode === 'hide' && isLabel(v)) continue;`).
   - Se `labelMode === 'endpoint'` : se il nodo sorgente del passo u è una label (e non è il seed stesso), il ciclo esegue `continue` , evitando di esplorare i vicini di u. La label compare quindi nell'intorno ma non fa da "ponte" verso altri nodi.
   - Se `labelMode === 'through'` : le label sono attraversate come normali nodi.
4. **Aggregazione Multi-Seed**:
   - **`hop` (Mappa `id -> distanza minima`)**: se un nodo v è raggiungibile da più seed a distanze diverse, `hop.get(v)` conserva la distanza minima assoluta.
   - **`reachedBy` (Mappa `id -> lista di seed`)**: memorizza l'elenco di tutti i seed che sono riusciti a raggiungere il nodo v entro il raggio stabilito. Questa informazione è fondamentale per determinare i **nodi in comune (bridges)**.
5. **Estrazione del Sottografo Indotto (`edges`)**:
   - Si scorrono tutti i nodi scoperti (`hop.keys()`).
   - Per ogni arco (u, v) presente nel grafo originale, se anche v fa parte dei nodi raggiunti (`hop.has(v)`), l'arco viene inserito nel sottografo (la condizione `u < v` garantisce che ogni arco non orientato sia contato una sola volta).

3. Calcolo del Focus e Percorsi Minimi verso i Bridge

Il calcolo del **Focus** determina quali elementi del grafo devono rimanere in primo piano e quali devono essere sfumati durante l'interazione (hover del mouse o selezione con click).

3.1 Risoluzione del Nodo Attivo

In [src/ui/canvas-renderer.js](file:///D:/Users/yyi30043/Desktop/VDI/src/ui/canvas-renderer.js):

```
const focusId = hover ?? selected;
const focus = focusSet(focusId);
```

Se il mouse è sopra un nodo (`hover`), quel nodo ha la precedenza; in alternativa si usa il nodo cliccato (`selected`).

3.2 La Funzione `focusSet`

La funzione `focusSet(id)` restituisce l'insieme di tutti gli ID dei nodi da evidenziare:

1. **Vicini Diretti**:
   Inizialmente include il nodo stesso e tutti i suoi vicini nel grafo correntemente visualizzato:

$$\text{direct} = \{id\} \cup \text{viewAdj}(id)$$

2. **Evidenziazione dei Percorsi verso i Bridge (Shortest Paths)**:
   Se il nodo attivo è un **nodo di partenza (seed)** e l'opzione `showBridges` è attiva:
   - Trova tutti i nodi ponte raggiunti da questo seed:

$$\text{bridgeTargets} = \{n \mid n \neq id \wedge \text{isCommon}(n) \wedge id \in n.by\}$$

   - Invoca la funzione `pathsToTargets(viewAdj, id, bridgeTargets)` in [src/core/neighborhood.js] (file:///D:/Users/yyi30043/Desktop/VDI/src/core/neighborhood.js#L78).

3.3 Come Funziona `pathsToTargets`

`pathsToTargets` esegue una BFS all'interno del sottografo visibile tracciando la mappa dei padri `prev` :

- Durante l'esplorazione, per ogni vicino v di u:

$$\text{prev}(v) = u$$

- Per ogni nodo target in `bridgeTargets` , risale la catena dei padri da t fino alla sorgente $from$:

```
let cur = t;
while (cur != null && !result.has(cur)) {
  result.add(cur);
  cur = prev.get(cur);
}
```

- **Risultato visivo**: Quando l'utente punta o seleziona un seed, l'interfaccia illumina non solo i vicini immediati, ma l'intera catena di connessioni che spiega **perché e come** quel seed si connette agli altri gruppi o artisti condivisi.

3.4 Memoizzazione e Rendering del Focus

- Per garantire 60 FPS durante i movimenti del mouse, `focusSet` adotta una cache a livello singolo (`focusMemo = { id, set }`).
- **Nodi fuori dal focus**: disegnati con trasparenza ridotta (`globalAlpha = 0.16`).
- **Archi fuori dal focus**: disegnati con trasparenza ridotta (`globalAlpha = 0.4`).
- **Archi del focus**: disegnati con spessore maggiorato ($1.7/k$), opacità al 100% e colore di evidenziazione `--edge-hi` .

4. Disegno Grafico delle Forme e degli Elementi

Tutto il rendering grafico è realizzato in HTML5 Canvas 2D nel modulo [src/ui/canvas-renderer.js] (file:///D:/Users/yyi30043/Desktop/VDI/src/ui/canvas-renderer.js).

4.1 Differenziazione Geometrica dei Nodi (`shape`)

Per rendere immediata la lettura del tipo di entità senza dipendere unicamente dal colore, a ciascun tipo è assegnata una specifica forma geometrica:

```
function shape(n, r) {
  ctx.beginPath();
  if (n.type === 'label') {
    // Agenzia: quadrato con angoli arrotondati
    const s = r * 0.92;
    ctx.roundRect(n.x - s, n.y - s, s * 2, s * 2, s * 0.28);
  } else if (n.type === 'group') {
    // Gruppo: diamante / rombo (quadrato arrotondato ruotato di 45°)
    const s = r * 0.98;
    ctx.save();
    ctx.translate(n.x, n.y);
    ctx.rotate(Math.PI / 4);
    ctx.roundRect(-s, -s, s * 2, s * 2, s * 0.28);
    ctx.restore();
  } else {
    // Artisti (male, female, person): cerchio classico
    ctx.arc(n.x, n.y, r, 0, TAU);
  }
}
```

Tipo di Entità	Forma Grafica	Implementazione Canvas
Label (Agenzia)	Quadrato con angoli smussati	`ctx.roundRect(x - s, y - s, 2s, 2s, radius)`
Group (Gruppo)	Diamante / Rombo	`ctx.rotate(Math.PI / 4)` + `ctx.roundRect` centrato
Male / Female / Person	Cerchio	`ctx.arc(x, y, r, 0, 2π)`

4.2 Dimensionamento Dinamico dei Nodi

Il raggio di ciascun nodo non è fisso, ma dipende dal grado di connettività originale:

$$\text{rad}(d) = \begin{cases} 10 & \text{se } d \text{ è un seed} \\ \min(3.2 + \sqrt{\deg} \cdot 0.85,\ 9) & \text{altrimenti} \end{cases}$$

L'uso della radice quadrata attenua le differenze estreme tra nodi con pochi collegamenti e megahub con centinaia di relazioni.

4.3 Indicatori Visivi e Anelli di Stato

Sopra o attorno alla forma base vengono applicati anelli di contorno aggiuntivi:

1. **Nodo di Partenza (Seed)**: Cerchio concentrico nero/inchiostro esterno di raggio $r + 3$ e spessore $2.2/k$.
2. **Nodo Selezionato**: Cerchio concentrico accentato (blu elettrico / lilla) di raggio $r + 6.5$ e spessore $2/k$.
3. **Nodi Ponte (Bridge)**:
   - Alone circolare dorato sfumato (`--bridge-halo`) di raggio $r + 5$.
   - Contorno dorato (`--bridge`) che ricalca esattamente la forma del nodo (`shape(n, r)` con spessore $1.8/k$).

4.4 Rendering dei Nomi e Anti-Cluttering

I testi dei nomi vengono disegnati con una tecnica a doppio passaggio per garantire la massima leggibilità su qualsiasi sfondo:

1. **Contorno (Halo)**: `ctx.strokeText` con colore di sfondo (`--bg`) e spessore $3.6/k$.

2. **Riempimento (Testo)**: `ctx.fillText` con colore dell'inchiostro (`--ink`).
3. **Filtro di visualizzazione dinamica**:
   Per evitare che la vista diventi illeggibile per sovrapposizione di etichette:
   - I nomi vengono sempre mostrati per: i seed, il nodo selezionato, e (se abilitato) per i bridge fino a un massimo di 40.
   - Tutti gli altri nomi compaiono solo se il livello di zoom $k \geq 2.2$, se il nodo fa parte del focus con $k > 0.8$, o se l'utente ha attivato la checkbox "Mostra tutti i nomi".

5. Filtri, Controlli e Gestione dello Stato

La gestione dello stato è centralizzata nel modulo [src/ui/state.js] (file:///D:/Users/yyi30043/Desktop/VDI/src/ui/state.js) tramite un'architettura **reattiva ad eventi (Publish/Subscribe)**.

5.1 I Filtri Disponibili

Filtro / Controllo	Tipo di Input	Effetto sull'Algoritmo / Visualizzazione
Partenze (`seeds`)	Ricerca / Chip / Doppio Click	Imposta l'insieme di sorgenti della BFS multi-seed.
Sfavoriti (`blocked`)	Pulsante Evita / Clic Destro	Esclude i nodi dalla BFS; i cammini che passerebbero da essi vengono troncati.
Distanza massima (`radius`)	Range Slider (1 - 4)	Profondità massima di scansione dei livelli della BFS.
Modalità Agenzie (`labelMode`)	Dropdown (`through` , `endpoint` , `hide`)	Regola l'espansione della BFS attraverso i nodi label.
Evidenzia nodi in comune (`showBridges`)	Checkbox booleana	Attiva/disattiva l'alone dorato e i percorsi minimi verso i bridge.
Criterio Bridge (`commonMode`)	Dropdown (`all` vs `two`)	`all` : raggiunto da **tutti** i seed; `two` : raggiunto da **almeno due** seed.
Mostra tutti i nomi (`showNames`)	Checkbox booleana	Forza la scrittura delle etichette testuali su tutti i nodi senza attendere lo zoom.

5.2 Calcolo dei Nodi in Comune (`isCommon`)

```
function isCommon(seed, reachedByList) {
  if (state.seeds.size <= 1 || seed) return false;
  const need = state.commonMode === 'all' ? state.seeds.size : 2;
  return reachedByList.length >= need;
}
```

Un nodo è considerato "ponte" solo se ci sono almeno 2 seed nell'esplorazione, non è un seed esso stesso, ed è stato raggiunto da un numero di partenze $\geq \text{need}$.

5.3 Protezione dal Sovraffollamento (`MAX_NODES`)

Se l'intorno calcolato supera 1500 nodi (`MAX_NODES`), lo store imposta lo stato `overflow` :

- Il canvas viene svuotato per evitare blocchi del browser o cali drastici di frame rate.
- Viene mostrato un avviso a schermo che suggerisce all'utente di ridurre il raggio o sfavorire gli hub ad alto grado.

5.4 Statistiche Differenziali

Lo store calcola due passaggi della BFS:

1. Con i nodi bloccati attivi (`res`).
2. Senza blocchi (`base`).
   La differenza $\max(0, \text{base} - \text{res.hop.size})$ fornisce in tempo reale il conteggio esatto dei **nodi rimossi grazie all'inserimento degli sfavoriti**, mostrando l'efficacia del filtraggio.

6. Algoritmo di Forza Utilizzato (Layout a Forze)

Il posizionamento automatico dei nodi nello spazio è governato da un motore di simulazione fisica basato su **D3-Force** (d3-force) configurato in [src/ui/canvas-renderer.js] (file:///D:/Users/yyi30043/Desktop/VDI/src/ui/canvas-renderer.js#L47-L57).

6.1 Equazioni e Forze Attive nella Simulazione

La simulazione modella i nodi come particelle fisiche sottoposte a un campo di forze risultante:

$$\vec{F}_{\text{totale}} = \vec{F}_{\text{repulsione}} + \vec{F}_{\text{molle}} + \vec{F}_{\text{collisione}} + \vec{F}_{\text{centripeta}}$$

La configurazione specifica utilizzata nel codice è:

```
const sim = d3.forceSimulation()
  .alphaDecay(0.035)
  .velocityDecay(0.35)
  .force('link', d3.forceLink()
    .distance(l => l.isLabel ? 70 : 34)
    .strength(l => l.isLabel ? 0.05 : 0.6))
  .force('charge', d3.forceManyBody()
    .strength(-80)
    .distanceMax(380))
  .force('x', d3.forceX(0).strength(0.035))
  .force('y', d3.forceY(0).strength(0.035))
  .force('collide', d3.forceCollide()
    .radius(d => rad(d) + 2))
```

Analisi dettagliata delle singole forze:

1. **Forza Repulsiva Elettrostatica (`forceManyBody - charge`)**:
   - `strength(-80)` : Simula una carica elettrostatica negativa tra tutti i nodi, respingendoli a vicenda secondo la legge di Coulomb $F \propto -1/r^2$.
   - `distanceMax(380)` : Taglia il calcolo delle forze per coppie di nodi a distanza superiore a 380 pixel. Questo previene il degrado prestazionale da $O(n^2)$ a distanze elevate e impedisce ai cluster lontani di influenzarsi a vicenda.
2. **Forza Elastica dei Collegamenti (`forceLink - link`)**:
   - Agisce come una molla elastica tra i nodi connessi da un arco.
   - **Distanza differenziata**:
     - Archi standard (tra artisti/gruppi): distanza target corta `34px` .
     - Archi verso agenzie (`isLabel`): distanza target allungata `70px` .
   - **Rigidità differenziata (`strength`)**:
     - Archi standard: `0.6` (molla rigida, mantiene uniti i membri dello stesso gruppo).
     - Archi verso agenzie: `0.05` (molla molto cedevole). Questa scelta evita che un'agenzia con 100 artisti comprima tutti i membri in un unico groviglio inestricabile al centro.
3. **Forza di Collisione Spaziale (`forceCollide - collide`)**:
   - Raggio di collisione: $\text{rad}(d) + 2$.
   - Impedisce la sovrapposizione fisica delle forme geometriche dei nodi, garantendo che i cerchi, i diamanti e i quadrati abbiano sempre almeno 2 pixel di spazio libero tra loro.
4. **Forza Centripeta / Gravitazionale verso l'Origine (`forceX` e `forceY`)**:

- Attrae dolcemente tutti i nodi verso l'origine coordinate $(0, 0)$ con intensità `0.035` .
- Evita che componenti connesse isolate o nodi debolmente legati si allontanino indefinitamente nello spazio.

6.2 Dinamica Temporale e Stabilità Visiva

- **Smorzamento della velocità (`velocityDecay: 0.35`)**: Applica un attrito d'ambiente che riduce la velocità a ogni passo del 35%, evitando oscillazioni caotiche.
- **Decadimento termico (`alphaDecay: 0.035`)**: Il parametro α scende gradualmente fino alla soglia minima, fermando la simulazione quando il layout raggiunge l'equilibrio meccanico.
- **Conservazione della memoria di posizione (`snapshot` & `pos`)**:
  Quando l'utente cambia parametri (es. aggiunge un seed o cambia raggio), il renderer memorizza in `pos` le ultime coordinate note (x, y) di ciascun nodo già presente.
  I nuovi nodi che entrano nell'intorno vengono inizializzati in prossimità dei loro vicini già posizionati:

  ```
  const nb = adj.get(n.id).map(v => viewIdx.get(v)).find(v => v && v.x !== undefined);
  n.x = (nb ? nb.x : 0) + (Math.random() - .5) * 40;
  n.y = (nb ? nb.y : 0) + (Math.random() - .5) * 40;
  ```

  Questo previene l'effetto "esplosione" e garantisce una continuità visiva fluida e naturale durante l'esplorazione.
- **Interazione diretta (Drag & Drop)**:
  Quando l'utente trascina un nodo, `fx` e `fy` (coordinate fisse) bloccano temporaneamente il nodo sotto al cursore, mentre la simulazione viene riattivata con `sim.alphaTarget(0.25).restart()` , permettendo al resto della rete di riadattarsi elasticamente attorno al nodo spostato.